

CPT102 Data Structures: Tutorial 3: The Comparable interface and Selection Sort algorithm

Thomas Selig

XJTLU

April 3, 2026

Contents of today's lecture

The main goal: **implementing Selection Sort**

- Notion of order.
 - Realised in Java by implementing the `Comparable` interface.
- Selection sort:
 - Overview;
 - Java implementation;
 - Examples.

Reminder: ordered sets

Given a set A , a **binary relation** on A is a mapping:

$\mathcal{R} : A \times A \rightarrow \{\mathbf{true}, \mathbf{false}\}$. We write $a_1 \mathcal{R} a_2$ when $\mathcal{R}(a_1, a_2) = \mathbf{true}$.

Definition

A (total) **order relation** on a set A is a binary relation $<$ satisfying the following two conditions:

- *Trichotomy*: For any pair $(a, b) \in A \times A$, exactly one of $a < b$, $a = b$, $b < a$ holds.
- *Transitivity*: For any triple (a, b, c) , if $a < b$ and $b < c$, then we have $a < c$.

We say that $(A, <)$ is an *ordered set*.

Order in Java

This works:

```
public static void main(String[] args) {  
    List<String> myList = new ArrayList<>(  
        List.of("Well", "Hello", "There")  
    );  
    Collections.sort(myList);  
    System.out.println(myList);  
}
```

- How does Java know which order to use to sort a list of strings?
- Because of this:

```
public class String implements Comparable
```

The Comparable interface

- Contains a single method: `compareTo()`.
- If a class implements `Comparable`, this means we can compare two instances of the class (objects).
- Very general property: implemented by a large number of classes, many of which have little/nothing else in common!
- Lists of objects implementing `Comparable` can be ordered according to the `compareTo()` implementation.
- When a class implements `Comparable`, it must fulfill the `Comparable contract`, see here.

The Comparable contract

```
Value thisValue;  
Value thatValue;  
int result = thisValue.compareTo(thatValue);
```

- If thisValue is **less than** thatValue, result will be negative.
 - This means that $\text{thisValue} < \text{thatValue}$ for some order relation $<$.
- If thisValue is **greater than** thatValue, result will be positive.
 - This means that $\text{thatValue} < \text{thisValue}$.
- If thisValue is **equal** to thatValue, result will be zero.
 - Here, we take “equal” to mean that neither of the two above comparisons holds, to guarantee trichotomy.

Our main topic for today: **selection sort**

- Idea is simple: remove minimal elements from the list one-by-one, appending them to the end of the sorted list.
- As with insertion sort, we implement an in-place version where the array is split into a sorted portion `arr[0 : i]` and an unsorted portion `arr[i : n]`.
 - For i ranging from 0 to $n - 2$, find the minimum element in `arr[i : n]` (minimum remaining unsorted element) and place it at index i .

Selection sort: implementation

Algorithm 1: Selection sort

```
 $n \leftarrow \text{arr.length};$   
for  $i \leftarrow 0$  to  $n - 2$  do  
     $\text{minIndex} \leftarrow i; \text{currentMin} \leftarrow \text{arr}[i];$   
    for  $j \leftarrow i + 1$  to  $n - 1$  do  
        if  $\text{LESS}(\text{arr}[j], \text{currentMin})$  then  
             $\text{minIndex} \leftarrow j; \text{currentMin} \leftarrow \text{arr}[j];$   
     $\text{SWAP}(\text{arr}, i, \text{minIndex});$ 
```

Selection sort: example

Iteration i	Array State (Before Swap)	Min Found in Unsorted	Swap Action	Array State (After Swap)
$i = 0$	[H, A, L, T, E, D]	1	SWAP(0, 1)	[A, H, L, T, E, D]
$i = 1$	[A, H, L, T, E, D]	5	SWAP(1, 5)	[A, D, L, T, E, H]
$i = 2$	[A, D, L, T, E, H]	4	SWAP(2, 4)	[A, D, E, T, L, H]
$i = 3$	[A, D, E, T, L, H]	5	SWAP(3, 5)	[A, D, E, H, L, T]
$i = 4$	[A, D, E, H, L, T]	4	SWAP(4, 4)	[A, D, E, H, L, T]

- The SWAP(4, 4) operation is trivial, but is still executed.
- The final unsorted element 'T' doesn't need sorting as it's already in the correct (only) place.

Selection sort: instability

Definition

A sorting algorithm is said to be **stable** if it preserves the relative order of any pair of equal elements from the original array.

Selection sort on input $[C^1, O, C^2, A]$

Iteration i	Array State (Before Swap)	Min Found in Unsorted	Swap Action	Array State (After Swap)
$i = 0$	$[C^1, O, C^2, A]$	3	SWAP(0, 3)	$[A, O, C^2, C^1]$
$i = 1$	$[A, O, C^2, C^1]$	2	SWAP(1, 2)	$[A, C^2, O, C^1]$
$i = 2$	$[A, C^2, O, C^1]$	2	SWAP(2, 3)	$[A, C^2, C^1, O]$

- Order of C^1 and C^2 is reversed in the final array, so selection sort is an **unstable** sorting algorithm.
- In contrast, bubble sort and insertion sort are **stable**.

Selection sort: complexity analysis

- Loops are fixed size for-loops, so the time complexity will be the same in best, worst and average cases.
- Comparisons:** One comparison per iteration of the inner loop; the outer loop has $n - 1$ iterations, the inner loop $n - i - 1$, giving:

$$\text{\#comparisons} = \sum_{i=0}^{n-2} (n - i - 1) = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2} = O(n^2)$$

- Swaps:** One swap per iteration of the outer loop, so:

$$\text{\#swaps} = n - 1 = O(n)$$

- Total time complexity is still $O(n^2)$ (quadratic).
- However**, if swaps are much more costly than comparisons, selection sort is a better choice than other algorithms seen in the lectures.
- Space complexity is still $M(n) = O(1)$ (constant, since in-place).

Coding time!

On Learning Mall, you will find a skeletal `Selection.java` class, containing all method headers.

- Start by completing the `less()` and `swap()` helper methods.
- Then complete the `sort()` method using the insertion sort algorithms.
- If time permits, complete the `isSorted()` methods, and use them for validation in the `sort()` method.

Happy coding!

Lab Exercise 1: sorting integer grades

- Create a new IntelliJ IDEA project called “Lab2”.
- Inside this project, create a class `GradeSorter.java` with a `main()` method.
- Inside this class, write a method `selectionSort(int[] grades)` which sorts the supplied array of grades in increasing order using the selection sort algorithm.
 - The logic should be as that coded in the `Selection.sort()` method of the tutorial.
 - There is no need to set up `less()` and `swap()` helper methods: their logic can be directly coded in `selectionSort()`.
- Test your `selectionSort()` method in `main()` by creating an array of grades, sorting it, and printing the array after sorting.
 - Remember that in Java, to print the contents of an array `arr`, you should write:
`System.out.print(Arrays.toString(arr));`

Solution 1: selectionSort()

```
3  ▶ public class GradeSorter {
4
5  @   public static void selectionSort(int[] grades) { 1 usage
6      int n = grades.length;
7      for (int i = 0; i < n; i++) {
8          int minIndex = i;
9          for (int j = i + 1; j < n; j++) {
10             // Find the index of the smallest score
11             if (grades[j] < grades[minIndex]) {
12                 minIndex = j;
13             }
14         }
15         // Swap the found minimum with the current position
16         int temp = grades[i];
17         grades[i] = grades[minIndex];
18         grades[minIndex] = temp;
19     }
20 }
```

Exercise 2: sorting arrays of custom objects

- In the same project, create a class `Selection.java` and copy the code from the “Solution for Tutorial 3” file on Learning Mall here.
 - Delete the `main()` method from this class, as it is unnecessary to have two main methods in the same program.
- Now create a `Student.java` class that represents students in a university. The `Student` class should store instance variables for the student's `name` (type `String`) and `gpa` (type `int`), and implement `Comparable` so that students are ordered according to their `gpa`.
- Finally, in the `main()` method of the `GradeSorter` class, test the sorting of `Student` objects.
 - Create different `Student` objects with different GPAs.
 - Create an array of `Student` objects which stores these `Student` objects.
 - Use the `Selection.sort()` method to sort the resulting array, and print the result using a for loop.

Try to write a test which illustrates the instability of selection sort, as details on Slide 9.

Solution 2(1): Student class

```
public class Student implements Comparable<Student> { 12 usages

    String name; 2 usages
    int gpa; 4 usages

    public Student(String name, int gpa) { 4 usages
        this.name = name;
        this.gpa = gpa;
    }

    @Override
    public int compareTo(Student o) {
        return Integer.compare(this.gpa, o.gpa);
    }
}
```

Solution 2(2): testing Selection.sort() in main()

// test for Exercise 2

```
Student s1 = new Student("Alice", 80);
```

```
Student s2 = new Student("Bob", 60);
```

```
Student s3 = new Student("Carol", 80);
```

```
Student s4 = new Student("David", 55);
```

```
Student[] students = {s1, s2, s3, s4};
```

```
Selection.sort(students);
```

```
for (Student s : students) {
```

```
    System.out.println("Student: name = " + s.name + ", gpa = " + s.gpa);
```

```
}
```